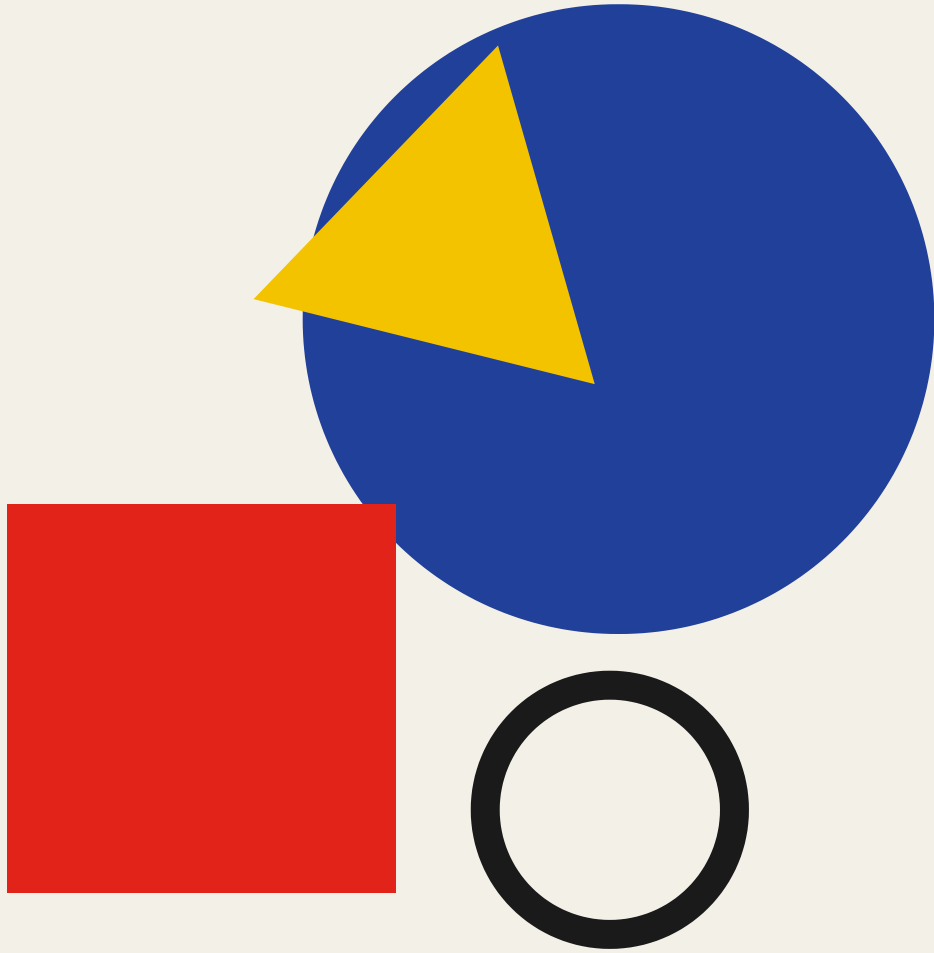


A FIELD GUIDE

vibe-code securely.

ship ai-built apps without getting rekt.

vibe-code securely: securing ai-generated apps before you ship

Who this is for: You're building apps with AI assistants (Cursor, Copilot, Claude, ChatGPT). You ship fast, and you're not a security engineer. You've heard the stories: exposed API keys, SQL injection through an AI-generated endpoint, a $50,000 cloud bill from an unsecured S3 bucket. This guide is a practical, copy-paste checklist for securing AI-generated code. Work through it before you ship.

---

1. secrets management

AI assistants frequently hardcode keys. They see `OPENAI_API_KEY=sk-...` in a tutorial and embed it directly in `config.py`. This is the most common catastrophic failure in AI-generated code.

the rule

No secret ever touches a source file. Not in `.py`, `.js`, `.ts`, `.env.example`, or the README. The moment a key is in a file that gets committed, assume it's compromised.

what to do

```
# ✅ Create a .env file (and NEVER commit it)
echo 'DATABASE_URL=postgresql://user:pass@localhost/db' >> .env
echo 'OPENAI_API_KEY=sk-abc123' >> .env

# ✅ Add .env to .gitignore IMMEDIATELY, before the first commit
echo '.env' >> .gitignore
echo '.env.*' >> .gitignore
echo '*.pem' >> .gitignore
```

In your code, use a library instead of hand-rolling:

```
# ✅ Python: python-dotenv
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")  # never the value directly
```

```
// ✅ Node.js: dotenv
require('dotenv').config();
const apiKey = process.env.OPENAI_API_KEY;
```

the pre-commit check

Run this before every commit. It catches keys that slipped past your `.gitignore`:

```
# Scan for accidental secrets in staged files
git diff --cached | grep -E '(sk-[a-zA-Z0-9]{20,}|AKIA[0-9A-Z]{16}|ghp_[a-zA-Z0-9]
{36}|-----BEGIN.*PRIVATE KEY)'
```

If it prints anything, **stop and fix it.** A committed secret is a compromised secret; rotate it immediately.

environment variables in production

For deployed apps, never hardcode env vars in Dockerfiles. Use your platform's secret manager:

```
# ✅ Docker Compose: secrets from host env
services:
  app:
    environment:
      - DATABASE_URL=${DATABASE_URL}   # NOT the actual URL
```

---

2. authentication

AI assistants generate auth code that *looks* right but is often wrong. They implement password resets with guessable tokens, sessions with no expiry, or "admin" checks done client-side.

don't roll your own auth

Use a battle-tested library.

```
# ✅ Node.js: Passport.js or NextAuth
npm install next-auth

# ✅ Python: Django's built-in auth or Flask-Login + Flask-Security
pip install flask-login flask-security-too

# ✅ For SPAs: Supabase Auth, Clerk, Auth0 (free tiers exist)
```

minimum auth checklist

- [ ] Passwords hashed with bcrypt/argon2 (NOT SHA-256, NOT MD5)
- [ ] Sessions expire server-side (not just cookie expiry)
- [ ] Password reset tokens are single-use, have a 15-minute expiry, and aren't sequential
- [ ] Rate-limit login attempts (5 failures = 15-minute lockout)
- [ ] Admin/privilege checks happen server-side. Never trust `isAdmin: true` from the client

client-side admin checks

AI codegen frequently produces this:

```
// ❌ insecure: anyone can open DevTools and change this
if (user.role === 'admin') {
  showAdminPanel();
}

// ✅ correct: check on the server
// The server validates the session token and returns only what that user can access.
```

---

3. input validation

AI models have no intuition for adversarial input. They will generate an endpoint that passes `req.body.query` directly into a SQL string and call it done. Audit every input.

sql injection

```
# ❌ injectable
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")

# ✅ parameterized queries
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

prompt injection

If your app takes user text and feeds it to an LLM, you have a prompt injection surface:

```
# ❌ user controls the system prompt
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": f"Translate: {user_input}"}
    ]
)

# ✅ separate system and user roles; untrusted text goes in 'user' only
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a translator. Translate the user's text
to French."},
        {"role": "user", "content": user_input}
    ]
)
```

command injection

```
# ❌ Never pass user input to shell
os.system(f"ffmpeg -i {filename} output.mp4")

# ✅ Use subprocess with argument arrays
subprocess.run(["ffmpeg", "-i", filename, "output.mp4"], check=True)
```

cross-site scripting (xss)

```
// ❌ innerHTML with user content = XSS
document.getElementById('comment').innerHTML = userComment;

// ✅ textContent (or sanitize with DOMPurify)
document.getElementById('comment').textContent = userComment;
```

the input audit loop

Every time AI generates an endpoint, ask three questions: 1. Does this input touch a database? Parameterize it. 2. Does this input touch a shell? Use argument arrays. 3. Does this input render in HTML? Escape or sanitize.

---

4. dependencies

AI assistants will `npm install` 800 packages for a to-do app. Every dependency is a trust decision.

lock it down

```
# ✅ Pin exact versions, no ranges
npm install express@4.18.2 --save-exact

# ✅ Audit regularly
npm audit
pip-audit       # pip install pip-audit first
```

auditing ai-added dependencies

If `package.json` has more than 30 dependencies and you can't explain what half of them do, you have a problem. AI agents import libraries they've seen in training data, not libraries you need. Before going to production:

```
# Find unused dependencies
npx depcheck
# Remove them
npm uninstall <package>
```

supply-chain hygiene

- [ ] Use `package-lock.json` / `yarn.lock` / `Pipfile.lock`, and commit them
- [ ] Pin exact versions (no `^` or `~` in critical deps)
- [ ] Run `npm audit` / `pip-audit` in CI, and fail the build on critical vulns
- [ ] Be skeptical of packages with < 100 weekly downloads or no recent commits
- [ ] Never `curl | bash` an install script without reading it first

---

5. safe deploys

environment separation

```
# ❌ hardcoded environment, debug in production
DEBUG = True
SECRET_KEY = "dev-secret-123"

# ✅ Environment-aware config
DEBUG = os.getenv("ENVIRONMENT") == "development"
SECRET_KEY = os.getenv("SECRET_KEY")  # fails loudly if missing
```

what ai forgets to disable in production

- [ ] Django `DEBUG = True` / Express `NODE_ENV=development`
- [ ] Debug toolbars (Django Debug Toolbar, React DevTools in prod build)
- [ ] `/graphql` introspection in production
- [ ] Directory listing on your web server
- [ ] Default admin credentials (`admin/admin`, `root/root`)
- [ ] Open `.git` directories (exposed via web server misconfiguration)

docker deploy checklist

```
# ✅ Use specific base images, not :latest
FROM node:20-alpine

# ✅ Run as non-root
USER node

# ✅ Multi-stage builds; don't ship dev dependencies
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
# ... build steps ...

FROM node:20-alpine
COPY --from=builder /app /app
USER node
CMD ["node", "dist/index.js"]
```

---

6. common ai codegen security mistakes

1. **Hardcoded secrets:** API keys, DB passwords, JWT secrets in source files
2. **SQL/NoSQL injection:** unparameterized queries from user input
3. **Missing auth on sensitive endpoints:** AI generates CRUD routes with no auth middleware
4. **Client-side authorization:** role checks in JavaScript, not on the server
5. **Debug mode in production:** verbose error pages leaking stack traces and paths
6. **Unpinned dependencies:** `^2.0.0` silently accepts breaking changes
7. **Leaking PII to logs:** AI logs request bodies containing emails, passwords, tokens
8. **No CORS policy:** `Access-Control-Allow-Origin: *` with credentials
9. **SSRF via URL inputs:** AI fetches `user_provided_url` with no validation
0. **Token leakage in URLs:** reset tokens in query strings (logged by proxies, analytics)

---

7. pre-ship security checklist

Before you go live, run this. Every check takes under 5 minutes.

```
☐  .env is in .gitignore and never committed (verify with `git log -- .env`, should
error)
☐  No API keys, passwords, or tokens in source files (grep for 'sk-', 'AKIA', 'ghp_',
'BEGIN.*KEY')
☐  All user input goes through parameterized queries or an ORM
☐  Auth checks happen server-side, not client-side
☐  Password resets use single-use, time-limited, non-sequential tokens
☐  Rate limiting on login, password reset, generation, and any expensive endpoints
☐  Production config: DEBUG=False, no debug toolbars, no default credentials
☐  Dependencies audited: `npm audit` / `pip-audit` returns zero criticals
☐  Docker runs as non-root user
☐  HTTPS enforced (redirect HTTP to HTTPS, HSTS header set)
☐  Backups: database backed up daily, backups tested by restoring them
☐  CORS restricted to your actual frontend domains (not '*')
☐  Error pages in production don't leak stack traces, paths, or DB errors
☐  No open admin panels at /admin, /api/admin, /graphql (introspection off)
☐  API keys scoped to minimum permissions (read-only where possible)
☐  A friend (not you, not the AI) has tried to break your app for 15 minutes
```

8. api security

AI-generated backends are good at creating endpoints and poor at protecting them. A vibe-coded API typically ships with every endpoint open, no rate limiting, and verbose errors that leak internal structure.

authentication on every sensitive route

```
# ❌ auth is optional or missing
@app.route('/api/users/<id>')
def get_user(id):
    return User.query.get(id).to_dict()

# ✅ Auth middleware on every route that touches data
@app.route('/api/users/<id>')
@require_auth                    # the check AI commonly omits
def get_user(id):
    if request.user.id != id and not request.user.is_admin:
        abort(403)
    return User.query.get(id).to_dict()
```

rate limiting

Without rate limiting, one script can hammer your `/api/login` endpoint 10,000 times per second, or one user can `POST /api/generate` and rack up a $500 AI bill.

```
# ✅ Flask rate limiter (pip install flask-limiter)
from flask_limiter import Limiter
limiter = Limiter(app, key_func=lambda: request.remote_addr)

@app.route('/api/login', methods=['POST'])
@limiter.limit("5 per minute")    # 5 attempts per IP per minute
def login():
    ...

@app.route('/api/generate', methods=['POST'])
@limiter.limit("20 per hour")     # prevents AI API bill runaways
def generate():
    ...
```

```
// ✅ Express rate limiter (npm install express-rate-limit)
const rateLimit = require('express-rate-limit');
const loginLimiter = rateLimit({
    windowMs: 60 * 1000,  // 1 minute
    max: 5,
    message: 'Too many login attempts, try again later'
});
app.post('/api/login', loginLimiter, loginHandler);
```

don't leak internals in errors

```
# ❌ default Flask error exposes file paths, line numbers, stack traces
# (this is the default behavior in debug mode)

# ✅ production error handler: safe message, log the details internally
@app.errorhandler(500)
def handle_500(e):
    app.logger.error(f'Server error: {e}', exc_info=True)  # logged, not shown
    return jsonify(error="An unexpected error occurred"), 500
```

cors

AI frequently sets `Access-Control-Allow-Origin: *` because it works without configuration. It also lets any website make authenticated requests to your API.

```
// ✅ Express CORS: whitelist your frontend domain
const cors = require('cors');
app.use(cors({
    origin: ['https://yourapp.com', 'https://staging.yourapp.com'],
    credentials: true
}));
```

---

9. incident response

If your app is compromised:

1. **Rotate every key and secret immediately:** assume they're all compromised
2. **Check your cloud bill:** crypto miners show up as sudden CPU spend
3. **Audit recent database changes:** look for new admin users, modified records
4. **Check your git history:** the attacker may have committed backdoors
5. **Revoke and reissue all auth tokens and sessions**
6. **Review server access logs:** find the entry point
7. **Fix the hole** before bringing anything back online
8. **Tell your users** if their data was exposed. Be honest and prompt.

10. scanning in ci

Install `trivy` or `snyk` and run it in CI. These scan your dependencies, Docker images, and IaC for known vulnerabilities. AI assistants rarely mention them, but they catch most supply-chain issues before they reach production.

```
# Scan a Docker image
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy image
yourapp:latest

# Scan a git repo for secrets
docker run --rm -v $(pwd):/src aquasec/trivy fs --scanners secret /src
```

These sections cover the most common security mistakes in AI-generated code. Print the pre-ship checklist in section 7 and run it before every launch.